

Name: Arijit Dutta
Enrollment Number: 12023002028077
Email: arijit.dutta.2023@iem.edu.in
Contact Number: 8777693763

Wipro Python Automation Capstone Assignment

Capstone Assignment 1

Automation of an E-Commerce Web Application using Selenium WebDriver with Python

1. Project Idea and Objective

This project automates the complete purchase journey of a customer on a public e-commerce demo application, using Selenium WebDriver with Python. The goal is to demonstrate, end to end, how a manual regression test for an online shopping flow can be converted into a repeatable, self-checking automation script that logs in, searches for a product, adds it to the cart, changes its quantity, verifies the cart contents, and produces a written record (screenshots + HTML report) of what happened.

Business Scenario

A customer wants to purchase a product from an e-commerce site. The site used for this assignment is the public OpenCart demo store at <https://tutorialsninja.com/demo/> (displayed under the name “Qafox.com”). The automation plays the role of that customer: it creates an account, logs in, searches for a product (“iPhone”), adds it to the shopping cart, updates the quantity, and confirms that the cart reflects the correct product, quantity and price before checkout.

What Needs to Be Done

The assignment specifies ten mandatory automation tasks. Each one is implemented as a distinct, clearly labelled step in the script:

1. Launch browser – start Chrome (or Firefox) and open the application URL.
2. Login to application – sign in with a valid, registered account.
3. Search product – use the site search box to look up a product by keyword.
4. Add product to cart – add the found product to the shopping cart.
5. Update quantity – change the quantity of the item already in the cart.
6. Verify cart details – confirm product name, quantity, unit price and totals in the cart.
7. Capture screenshots – save a PNG screenshot after every major step.
8. Read test data from Excel/JSON – load names, credentials and product data from an external file instead of hard-coding it.
9. Handle popup/alerts – accept any native browser alert and dismiss the site's success/error banners.
10. Generate execution report – write a PASS/FAIL summary with screenshots to an HTML file after the run.

Tools and Technologies Used

- Python 3.9+ – scripting language
- Selenium WebDriver – browser automation library
- webdriver-manager – downloads the matching ChromeDriver / GeckoDriver automatically
- openpyxl – reads the Excel (.xlsx) test-data file
- json (standard library) – reads the JSON test-data file
- Chrome / Firefox browser – execution target
- HTML – hand-written execution report generated by the script
- Application under test: <https://tutorialsninja.com/demo/> (OpenCart demo store)

2. Project File Structure

The project is organised as a small, flat Python package so that configuration, reusable logic and the test flow itself stay separate:

```
capstone_project/
|-- main.py           run this file - the purchase flow, step by step
|-- settings.py       browser, data source, URL, waits, folder paths
|-- helpers.py        reusable functions used by main.py
|-- requirements.txt  packages to install
|-- README.md         project documentation
|-- video_script.txt  line-by-line explanation script for the demo video
|-- data/
|   |-- test_data.json test data (JSON version)
|   |-- test_data.xlsx same test data (Excel version, sheet "TestData")
|-- screenshots/      created on the first run, one PNG per step
|-- reports/          created on the first run, one HTML report per run
```

3. What Each File Does

settings.py

Holds every value that might change – nothing executes in this file. It defines the browser to use, the data source (JSON or Excel), the base URL, wait/pause timings, and the folder paths for data, screenshots and reports. Paths are built from the file's own location, so the project runs correctly from any working directory.

helpers.py

Contains the small, reusable functions that main.py calls. Each function maps to one assignment task:

Function	Task #	What it does
read_test_data()	8	Reads the JSON or Excel file (based on settings.DATA_SOURCE) and returns one dictionary of test data.
start_browser()	1	Starts Chrome or Firefox via webdriver-manager, maximizes the window and sets the implicit wait.
take_screenshot()	7	Saves a timestamped PNG into screenshots/ and records its name for the report.
handle_alert_if_present()	9	Checks for and accepts a native browser alert if one is open.
close_banner()	9	Closes the green/red status bar shown after cart actions so it doesn't cover later screenshots.

Function	Task #	What it does
price_to_number()	–	Converts a displayed price string such as "\$1,202.00" into the number 1202.0 for comparison.
log_result()	10	Records one step's name, PASS/FAIL status and a note, for the final report.
write_report()	10	Writes the collected results and screenshots into a formatted HTML report inside reports/.

main.py

The entry point of the project (run with “python main.py”). It contains a single main() function that carries out the ten tasks in order, with a clearly marked “TASK n” comment block for each one. The whole flow is wrapped in try / except / finally: if any step fails, the except block records which step broke and takes a failure screenshot, while the finally block always closes the browser and writes the execution report – so a report is produced whether the run passes or fails.

requirements.txt

Lists the Python packages the project depends on (selenium, webdriver-manager, openpyxl), so the environment can be set up with a single “pip install -r requirements.txt” command.

data/test_data.json and data/test_data.xlsx

Two equivalent copies of the same test data – one in JSON, one in Excel – so the script can demonstrate reading test data from either format, controlled by the DATA_SOURCE setting.

Field	Meaning
first_name, last_name, telephone, password	used to register the new customer and then log in
email_template	email address template; {stamp} is replaced with the current date/time on every run
search_keyword	text typed into the site's search box
product_name	exact product title expected in the search results and the cart
new_quantity	the quantity the script sets in the cart (2 or more, to make the update visible)

video_script.txt

A line-by-line narration script used to record a walkthrough video of the project, explaining what each function does and what appears on screen while it runs.

README.md

Project documentation: setup instructions, file structure, a task-to-code mapping table, and notes on output locations – the same information this report expands on.

4. Where Each Assignment Task Is Implemented

#	Task	Where
1	Launch browser	start_browser() in helpers.py, called from main.py
2	Login	“TASK 2” block in main.py
3	Search product	“TASK 3” block in main.py
4	Add product to cart	“TASK 4” block in main.py
5	Update quantity	“TASK 5” block in main.py
6	Verify cart details	“TASK 6” block in main.py
7	Capture screenshots	take_screenshot(), called after every step
8	Read test data	read_test_data() in helpers.py
9	Handle popups / alerts	handle_alert_if_present(), close_banner()
10	Execution report	write_report() in helpers.py

Note: the demo application has no ready-made shared login, so the script also performs a one-time setup step – registering a brand-new customer (with an email built from the current timestamp) and logging out – before the actual “Task 2: Login” step signs back in with that same account. This is reflected in the screenshots that follow.

5. Step-by-Step Execution Report

The section below walks through one full, successful run of main.py against https://tutorialsninja.com/demo/, using the actual screenshots captured by take_screenshot() during execution. Every screenshot file name begins with the step number so the order matches the script.

Step 1: Launch Browser and Open the Home Page

Chrome is started by start_browser() and navigated to the base URL. The script asserts that the page title contains “Your Store” before continuing, then saves “01_home_page.png”. The home page of the Qafox.com demo store loads successfully with the category menu, search box and empty shopping cart (“0 item(s) - \$0.00”) visible.

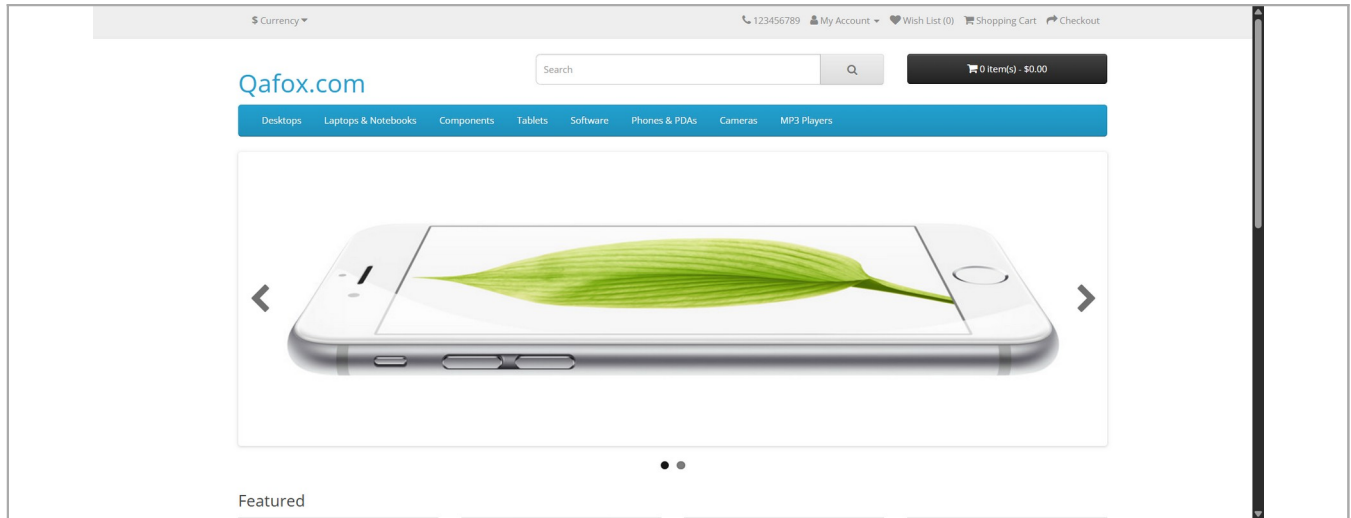


Figure 1 – Home page of the application under test (01_home_page.png)

Step Setup: Register a New Customer

Because the demo site does not provide a ready-made login, the script first registers a new account so that Task 2 (Login) has valid credentials to use. The registration form is filled in with the first name, last name, a timestamped e-mail address, telephone number and password read from the test data file, the privacy-policy checkbox is ticked, and “Continue” is clicked. The screenshot shows the completed form just before submission.

A screenshot of the registration form on the Qafox.com demo site. The form is filled with test data: First Name (Capstone), Last Name (Tester), E-Mail (capstone.test@20260927172952@example.com), Telephone (9876543210), Password (*****), and Password Confirm (*****). The Newsletter section has the "No" option selected. A checkbox for "I have read and agree to the Privacy Policy" is checked. A "Continue" button is visible at the bottom right of the form. The footer contains four columns of links: Information (About Us, Delivery Information, Privacy Policy), Customer Service (Contact Us, Returns, Site Map), Extras (Brands, Gift Certificates, Affiliate), and My Account (My Account, Order History, Wish List).

Figure 2 – Registration form filled with test data (02_registration_form.png)

After submitting the form, the site confirms account creation with the heading “Your Account Has Been Created!”, which the script asserts before moving on and logging out again.

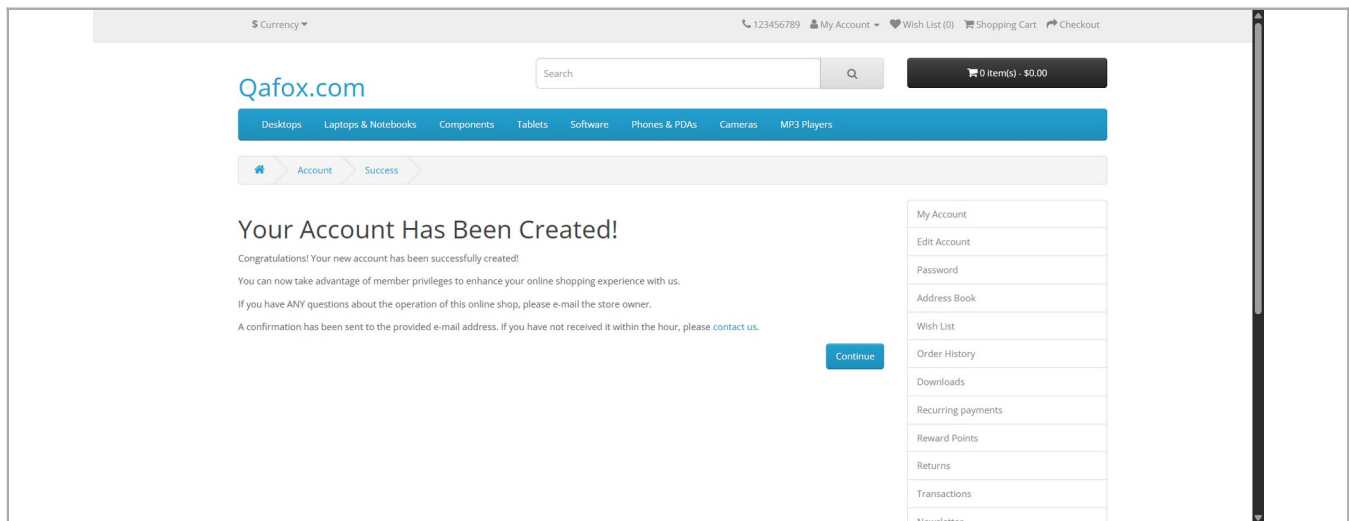


Figure 3 – Account creation confirmation page (03_account_created.png)

Step 2: Login

With the new account's e-mail and password, the script opens the login page, enters the credentials and clicks “Login”. The screenshot captures the login form just before submission, with the e-mail and password fields populated.

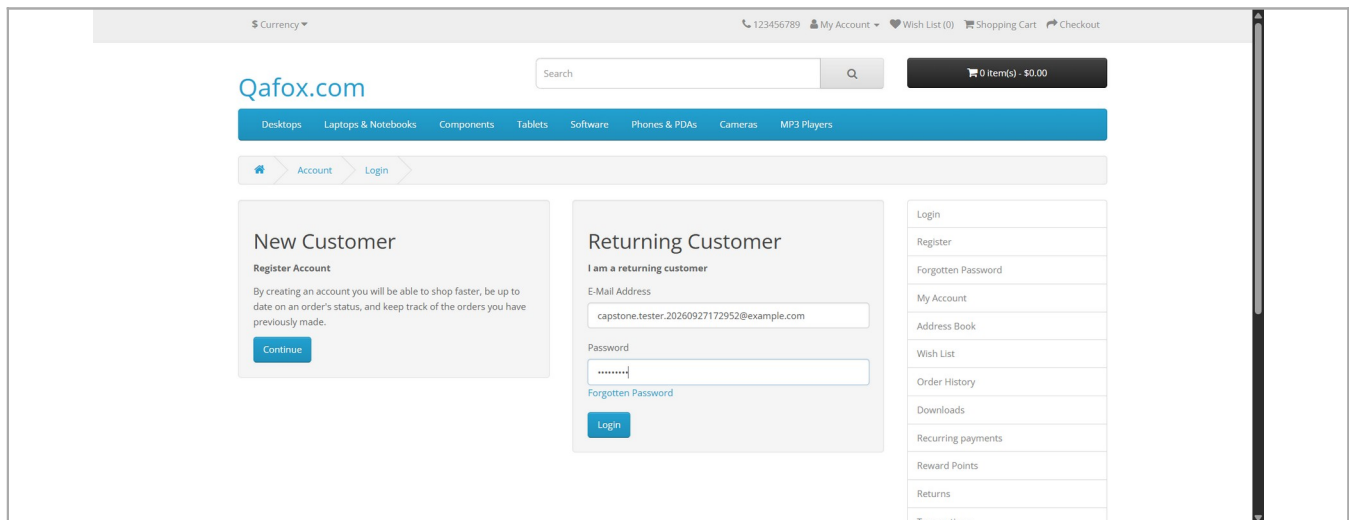


Figure 4 – Login page with credentials entered (04_login_page.png)

A successful login lands on the account dashboard showing the “My Account” heading with links such as “Edit your account information” and “View your order history” – the script asserts this heading is present before continuing.

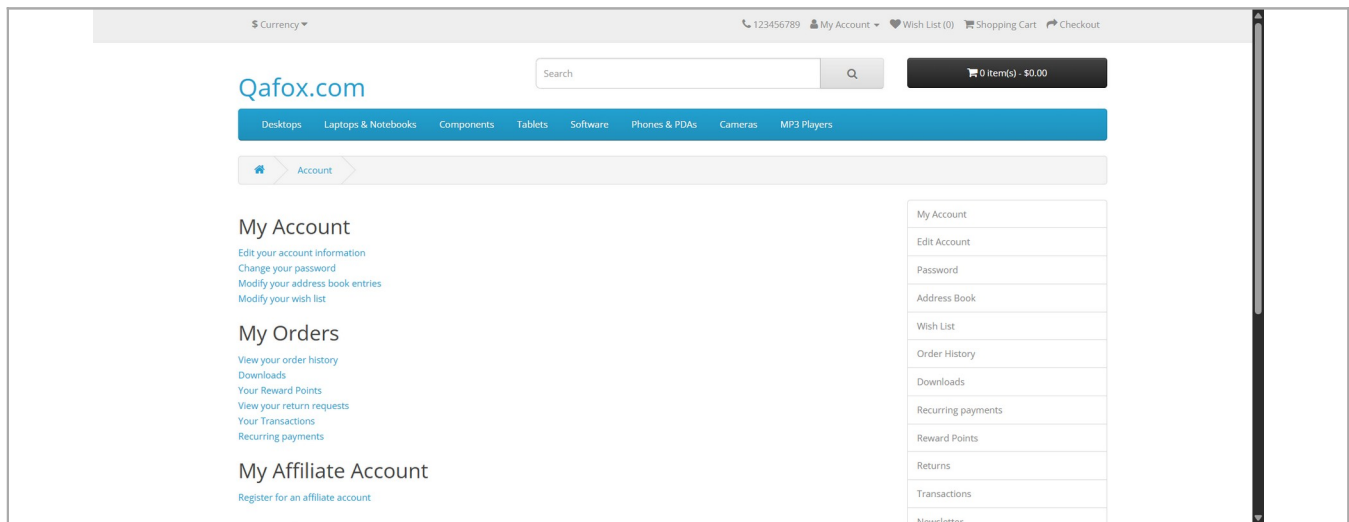


Figure 5 – My Account dashboard after a successful login (05_logged_in.png)

Step 3: Search Product

From the home page, the keyword “iPhone” (read from the test data) is typed into the search box and the search button is clicked. The script asserts that a product link matching the expected product name appears in the results before taking the screenshot.

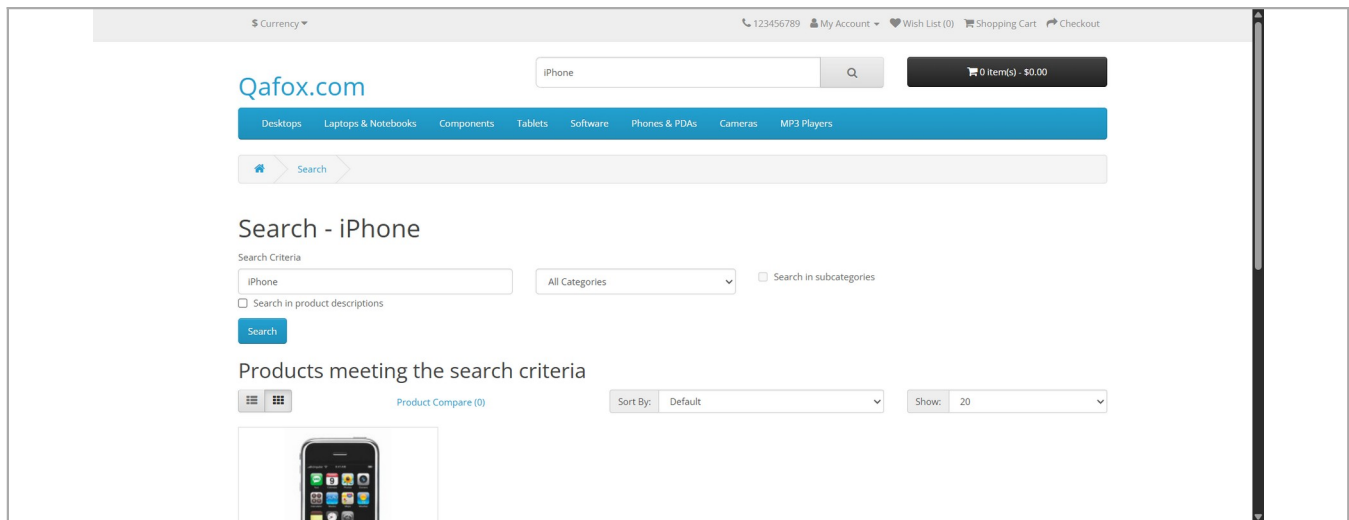


Figure 6 – Search results for “iPhone” (06_search_results.png)

Step 4: Add Product to Cart (with Popup Handling)

The “Add to Cart” button belonging to the iPhone product card is located and clicked. The script first checks for a native browser alert with `handle_alert_if_present()` and accepts it if one appears (Task 9), then asserts that the green success banner “Success: You have added iPhone to your shopping cart!” is shown, and the header cart total updates to “1 item(s) - \$123.20”.

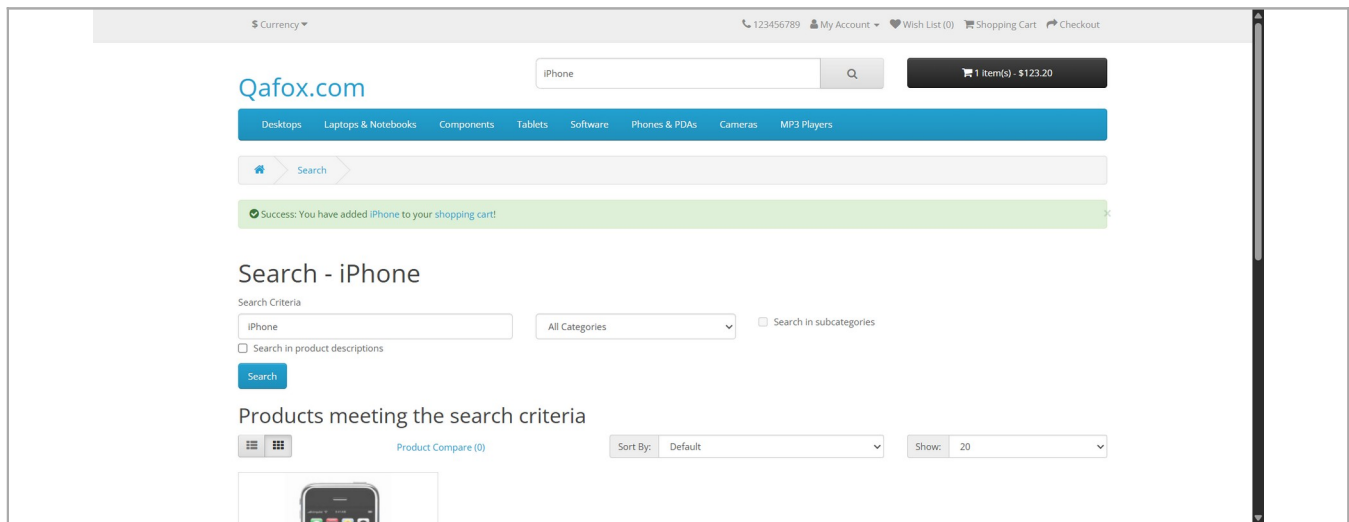


Figure 7 – Success banner after adding the product to the cart (07_added_to_cart.png)

Step 5: Update Quantity

On the Shopping Cart page, the quantity field for the iPhone line item is cleared and set to the new quantity from the test data (3), then the refresh/update button next to it is clicked. Any resulting alert is again handled automatically. The script asserts that the update banner (“...modified your shopping cart!”) appears, and the cart header total updates to “3 item(s) - \$369.60”. The red banner underneath is the site's own stock-level notice and does not affect the assertions.

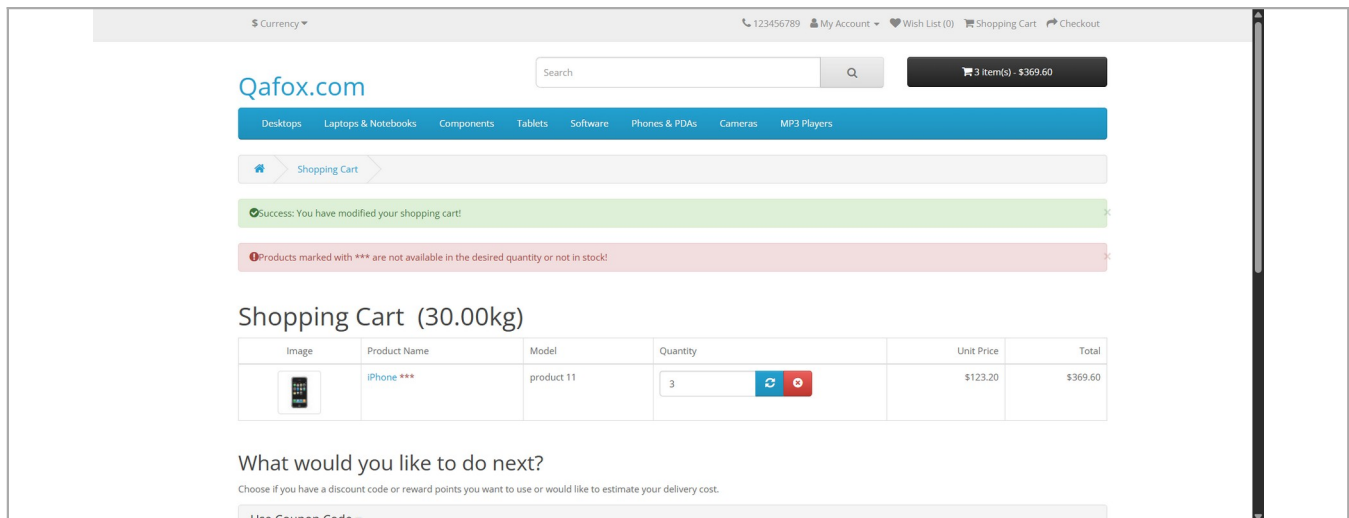


Figure 8 – Cart after the quantity is updated to 3 (08_quantity_updated.png)

Step 6: Verify Cart Details

The cart page is reloaded so the script checks what the server actually saved, not just what the page showed right after the click. It reads the product name, quantity, unit price and line total directly from the cart table, plus the sub-total from the totals block, and asserts that: the product name matches, the quantity equals 3, unit price × quantity equals the line total, and the line total equals the sub-total ($\$123.20 \times 3 = \369.60). All four checks pass, confirming the cart is in the expected state.

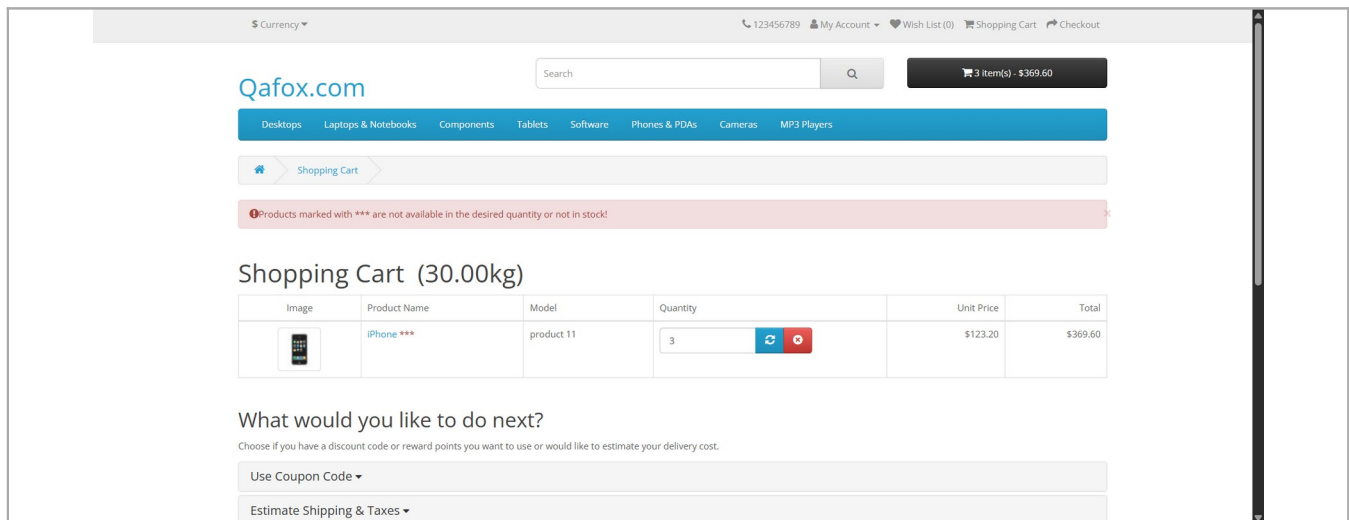


Figure 9 – Final verified cart: iPhone, quantity 3, total \$369.60 (09_cart_verified.png)

6. How Screenshots, Test Data, Alerts and Reporting Were Handled

Task 7 – Capture Screenshots

take_screenshot() is called after every step shown above (plus once more if a step ever fails), saving a PNG named “HHMMSS_<step>.png” into the screenshots/ folder. The nine screenshots in this report are exactly the files produced by one such run.

Task 8 – Read Test Data from Excel/JSON

read_test_data() is the very first thing main() does. Depending on settings.DATA_SOURCE, it loads either data/test_data.json or data/test_data.xlsx (sheet “TestData”) and returns a single dictionary that every later step reads from – names, e-mail template, password, search keyword, product name and the new quantity. Because the demo site rejects an e-mail address that is already registered, the e-mail is built from the template with the current timestamp inserted, guaranteeing a fresh account on every run.

Task 9 – Handle Popups / Alerts

Two kinds of “popup” are handled: handle_alert_if_present() checks for and accepts a native browser alert() dialog immediately after the add-to-cart and update-quantity clicks (the demo site does not normally raise one, so the check is defensive); close_banner() dismisses the green/red status bar the site shows after cart actions, so it does not obscure the page in later screenshots.

Task 10 – Generate Execution Report

write_report(), called from the finally block, always runs – whether the test passed or failed – and writes reports/execution_report_<date_time>.html. The report lists every step recorded by log_result() with a PASS/FAIL badge and a note (for example the final total price), followed by all screenshots taken during the run, giving a single self-contained record of the execution.

7. Execution Summary

Step	Result	Notes
8. Read test data	PASS	source: json, product: iPhone, new quantity: 3
1. Launch browser	PASS	title: Your Store
Setup. Register new customer	PASS	new account created with a timestamped e-mail
2. Login	PASS	“My Account” heading confirmed
3. Search product	PASS	searched for: iPhone
4. Add product to cart	PASS	success banner shown
9. Handle popup/alert	PASS	no browser alert appeared, banner only
5. Update quantity	PASS	quantity set to 3
6. Verify cart details	PASS	iPhone x 3, total \$369.60

All ten assignment tasks completed successfully in this run, and the final cart state (iPhone × 3, \$369.60) matches the test data supplied in data/test_data.json.

8. Conclusion

This capstone assignment demonstrates a complete, data-driven Selenium WebDriver automation of a realistic e-commerce purchase flow: launching a browser, registering and logging in a user, searching for a product, adding it to the cart, updating its quantity, and verifying the final cart state against calculated totals. The project separates configuration (settings.py), reusable logic (helpers.py) and the test flow itself (main.py), reads its test data externally from JSON/Excel rather than hard-coding it, defends against popups and alerts, and always produces evidence of the run in the form of screenshots and an HTML execution report – satisfying all ten tasks required by the assignment.